Eric Sohel

Stony Brook, NY | ericsohel05@gmail.com | linkedin.com/in/eric-sohel | github.com/ericsohel | ericsohel.com

Education

Stony Brook University — Stony Brook, NY
B.S. in Computer Science & Applied Mathematics — *Expected May 2027*

- **Coursework:** Data Structures, Algorithms, Systems (C/Unix), Software Development, Functional Programming (OCaml), Computer Networks, Discrete Math, Linear Algebra, Probability & Statistics

Experience

Capital One — New York, NY
Software Engineer Intern (Returning) — Low-Latency Platform Team — *Jun 2026 – Aug 2026*

- Returning to build real-time payment infrastructure under strict **p99 latency** budgets and **exactly-once** processing semantics.

Capital One — McLean, VA
Software Engineer Intern — Wire Transfer Platform — *Jun 2025 – Aug 2025*

- Built Capital One's **first in-house wire transfer platform** for **high-value daily payments** on internal AWS infrastructure — replacing the third-party vendor, eliminating per-transaction fees, and pulling money movement fully inside Capital One's security perimeter.
- Built **Node.js / Fastify** microservices on **AWS Lambda** and **Fargate** with idempotency keys and exponential-backoff retries — preventing duplicate debits under network jitter, upstream timeouts, and client retries.
- Designed an **Aurora PostgreSQL** approval **state machine** spanning 4 verification APIs for auditable multi-step clearance, with automated compliance checks and clean recovery from mid-flow worker crashes.
- Diagnosed and fixed a Postgres connection-pool saturation under peak load (**15 wires/sec**, 90% pool utilization) by releasing connections during outbound API calls — **p99 latency −40%**.

Meta (via MLH Fellowship) — Remote
Software Engineer Fellow — Pysa Static Analyzer — *May 2024 – Aug 2024*

- Contributed to **Pysa** within Meta's `facebook/pyre-check` repo — interprocedural taint analysis catching SQLi, SSRF, code injection, and path traversal across Instagram's Python codebase.
- Designed and shipped a **differential-testing harness** (merged as pyre-check#886) synthesizing **10,000+** Python programs with ground-truth source→sink labels — randomized call graphs, decorator stacks, and API-misuse patterns — to grade Pysa's analysis against an oracle.
- Surfaced **20+** previously-undetected exploit-shape taint flows acknowledged by the maintainer; locked each as a regression test.
- Merged **4** PRs to `facebook/pyre-check` (registry instrumentation, beginner-friendly diagnostic warnings, documentation, plus the fuzzer above).

Stanford Code in Place (CS 106A) — Remote
Section Leader — *Mar 2024 – Jun 2024*

- Led 2 weekly Python and Karel-the-Robot sections (∼20 students) for Stanford's free global **CS 106A** (Profs. Sahami & Piech) — **30,000+** learners across **100+** countries.
- Invited back as **Head Teaching Assistant** for the next cohort, mentoring incoming section leaders.

Projects

DraftIQ Player Data API | *Node.js, Express, SQLite, MLB Stats API* — Apr 2026 – May 2026

- Built a production REST API ingesting MLB data on a **tiered scheduler** (30 min injuries with active-hours gating, 6 hr depth charts & transactions, daily metadata, seasonal stats); idempotent jobs with exponential backoff and staleness-aware metadata.
- Designed an algorithmic valuation engine using **z-score-based valuation** against a replacement-level baseline with positional-scarcity adjustments and live draft-state re-valuation — **600+** player valuations in tens of milliseconds, feeding ranked nominations, positional-need scores, and budget allocations.
- Shipped a self-serve developer portal with full API-key management — **scrypt** password hashing, signed-cookie sessions, per-key **CIDR** whitelisting (pure-Node, no library), tumbling-window rate limiting, and a 30-day audit log.
- Authored **60+** Jest/supertest unit, integration, and state-transition tests; published an **OpenAPI 3.0** spec validated by CI to prevent route drift; deployed via **GitHub Actions** to Render + Vercel; live at player-data-api.onrender.com.

Technical Skills

Languages: Python, JavaScript / TypeScript, Java, SQL; familiar with C/C++, OCaml
Cloud & Distributed Systems: AWS (Lambda, Fargate, EC2, Aurora, SQS), Microservices, Idempotency, Exactly-once semantics, State machines, Exponential backoff, REST / OpenAPI 3.0
Databases: PostgreSQL, MongoDB, SQLite, Redis
Backend & Tooling: Node.js, Fastify, Express, React, Docker, Linux, Git, GitHub Actions, CI/CD, Jest, Static analysis

Awards & Competitions

Citadel Terminal Competition — 2nd place · **Cornell Trading Competition** — top 5 (2×) · **AIME Qualifier** (2×, top ∼5% of AMC test-takers)